VS Code & Co-Pilot: Vom Anfänger zum KI-Entwicklungsprofi

Werden Sie in nur 10 Stunden vom Git-Neuling zum VS Code- & Copilot-Champion. Mit KI-gestütztem Lernen revolutionieren wir Ihren Entwicklungsworkflow!



Ihr Weg zum KI-gestützten Entwickler

10 Module in 10 Stunden

Kompakter Lernpfad mit einem Modul pro Stunde - perfekt strukturiert für maximalen Lerneffekt.

Praxisnahe Übungen

Live-Coding, Break-out-Sessions und interaktive Quizzes festigen Ihr Wissen sofort.

Team-Kollaboration

Arbeiten Sie in einem von 10 Teams mit eigenem GitHub Repository an realen Projekten.

Zertifizierter Abschluss

Krönen Sie Ihren Lernerfolg mit einem Abschlussprojekt und erhalten Sie Ihr offizielles Zertifikat.





Unser einzigartiges KI-First Konzept

Code von Copilot & Co. generiert

Sämtlicher Code in allen Demos wird von KI-Tools wie GitHub Copilot erzeugt. Sie lernen nicht nur zu coden, sondern die KI optimal für Ihre Entwicklungsarbeit einzusetzen.

KI-Code reviewen statt selbst tippen

Entwickeln Sie die kritische Fähigkeit, KI-generierten Code zu verstehen, zu bewerten und zu verbessern - eine Schlüsselkompetenz für moderne Entwickler.

Prompt-Engineering für Entwickler

Meistern Sie die Kunst, präzise Anweisungen an KI-Tools zu formulieren, um genau den Code zu erhalten, den Sie benötigen.

Ihre Workshop-Ausstattung

Grundausstattung

VS Code, Git und optional GitHub Desktop - alles kostenlos verfügbar und einfach einzurichten.

Essentielle Erweiterungen

GitHub Copilot, VS Code Live Share und GitLens für maximale Produktivität und Zusammenarbeit.

Kommunikationskanäle

Zoom für den Hauptkurs, Gamma Live Polls für Feedback und ein Discord-Backchannel für schnelle Hilfe.

Unsere 10 Teams im Überblick

Team-Struktur

10 Teams (Alpha bis Juliet) mit je 5 Teilnehmern für optimale Lernerfolge und gegenseitige Unterstützung.

Ice-Breaker

60-Sekunden Selbstvorstellung zu Beginn für ein schnelles Kennenlernen aller Teilnehmer.



Projekt-Repositories

Jedes Team erhält ein vorkonfiguriertes GitHub Repository-Template als Ausgangsbasis.

Slack-Wars-Gamification

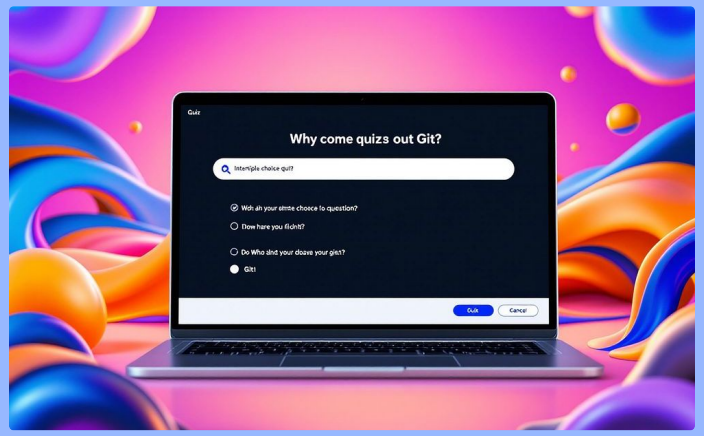
Sammeln Sie Badges und Achievements durch erfolgreiche Aufgabenlösung und Teamarbeit.

Interaktiver Einstieg mit Kahoot



QR-Code zur Teilnahme

Scannen Sie diesen Code mit Ihrem Smartphone, um sofort am interaktiven Warm-up Quiz teilzunehmen.



10 Git-Basisfragen

Testen Sie Ihr Vorwissen zu Git-Grundlagen in einem spannenden Wettbewerb gegen die anderen Teams.



Gewinnerteam-Privileg

Das siegreiche Team darf das erste Demo-Repository für unsere praktischen Übungen auswählen.

Modul 1: Git Grundlagen verstehen

Git's Snapshot-Prinzip

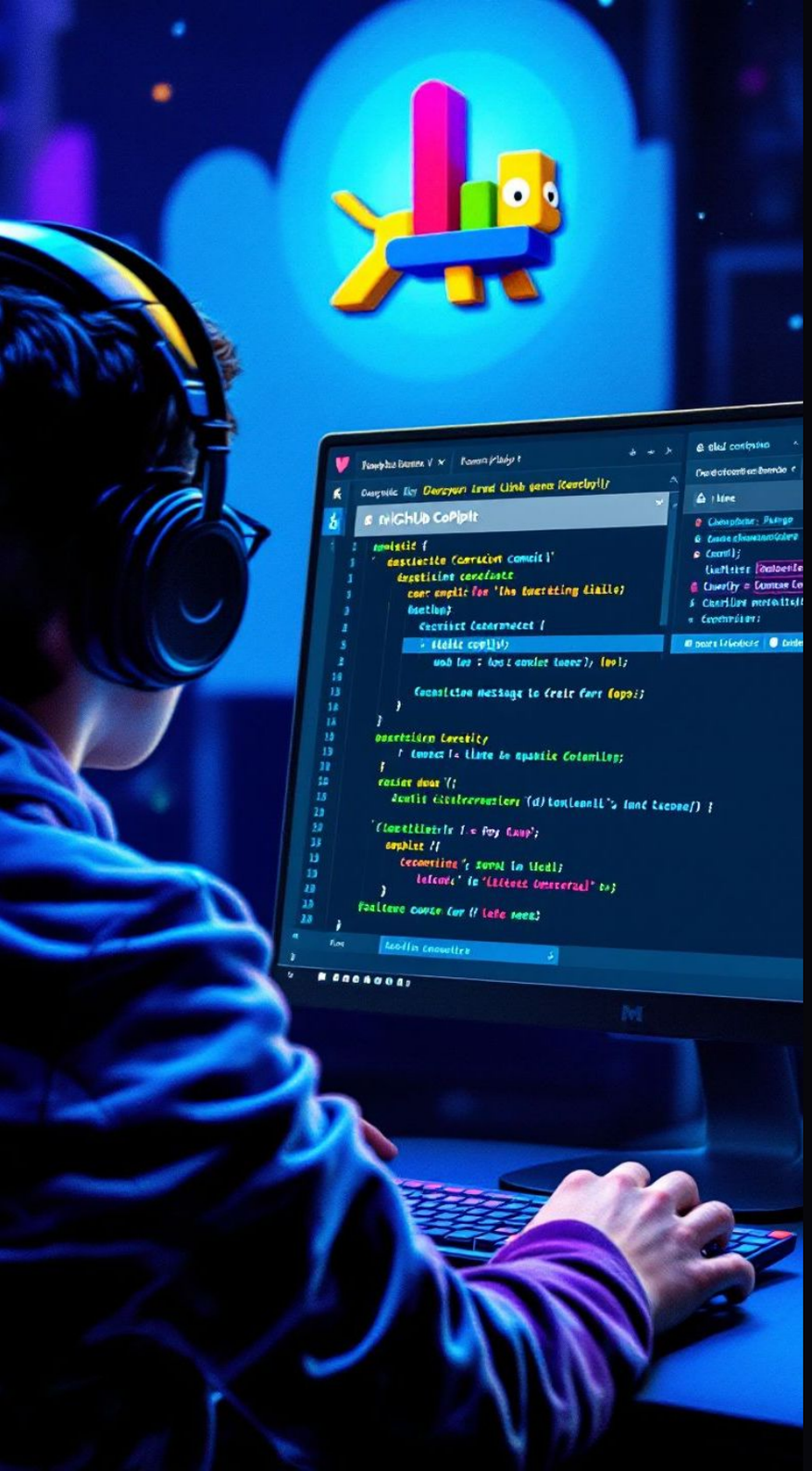
Git speichert den gesamten Zustand Ihres Projekts als Schnappschuss bei jedem Commit - nicht nur die Unterschiede zwischen Dateien. Diese Herangehensweise macht Git besonders robust und ermöglicht präzises Versionieren.

Diese fundamentale Eigenschaft unterscheidet Git von älteren Versionskontrollsystemen und bildet die Grundlage für seine Leistungsfähigkeit.

Traditionelle File-Diffs

Klassische Versionskontrollsysteme speichern nur die Unterschiede (Diffs) zwischen Dateiversionen. Das spart zwar Speicherplatz, führt aber bei komplexen Änderungen oft zu Problemen.

Im Gegensatz dazu kann Git jederzeit den exakten Zustand Ihres Projekts zu jedem Zeitpunkt wiederherstellen, was besonders in großen Teams wertvoll ist.



Praktische Übung: Ihr erstes Commit mit Copilot

Repository initialisieren

Lassen Sie Copilot Chat den Befehl zur Erstellung eines neuen Git-Repositories generieren und führen Sie ihn aus.

Erste Datei erstellen

Bitten Sie Copilot, eine einfache README.md Datei mit einer Projektbeschreibung zu generieren.

Änderungen committen

Nutzen Sie Copilot Chat, um den korrekten Git-Befehl zum Hinzufügen und Committen der neuen Datei zu erhalten.

Commit-Historie anzeigen

Verwenden Sie Copilot, um den Git-Befehl für die Anzeige Ihrer Commit-Historie zu generieren und zu verstehen.

Effiziente Branching-Strategie für große Teams

Feature-Branches

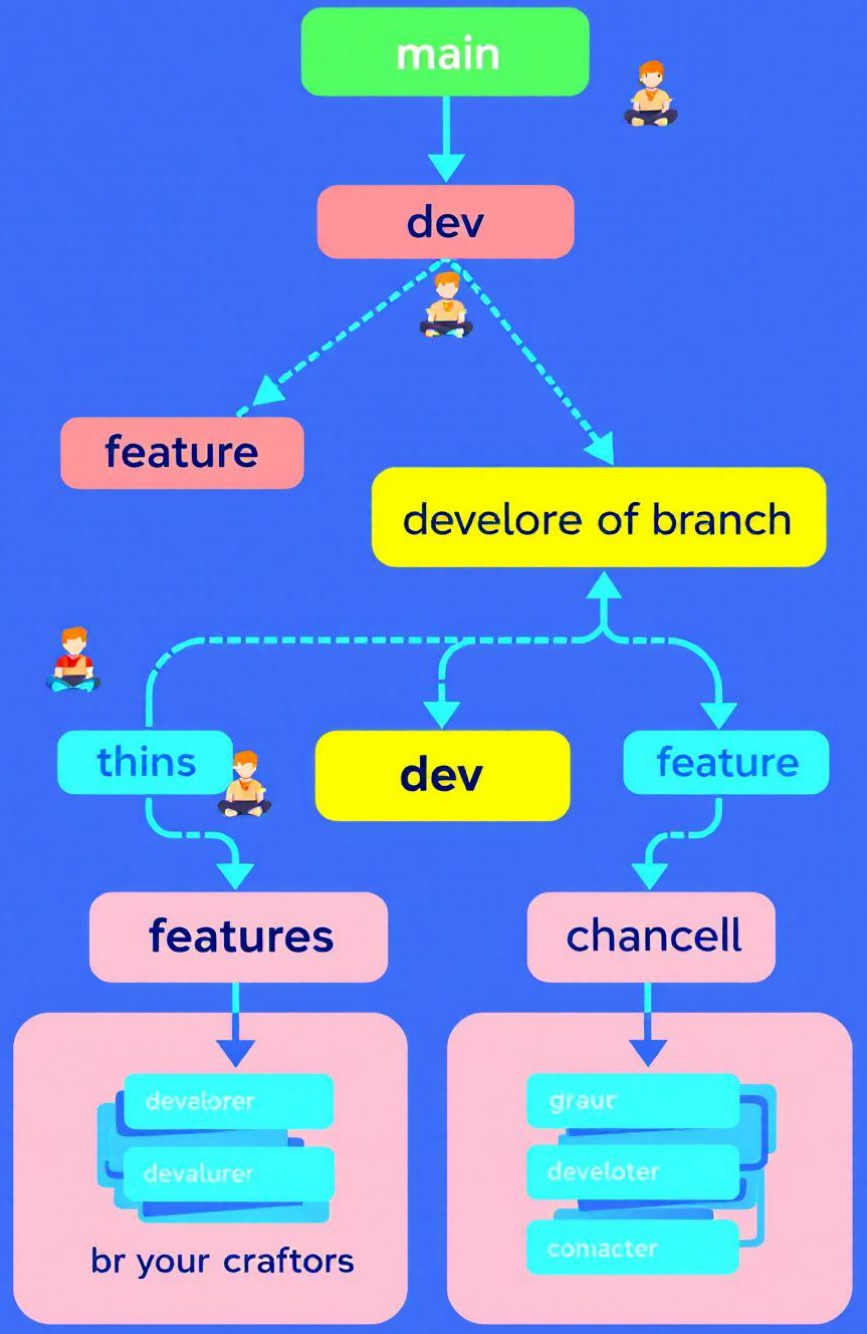
Jeder Entwickler arbeitet in eigenen Feature-Branches, isoliert von anderen Änderungen. Ideal für parallele Entwicklung in großen Teams.

Development-Branch

Fertige Features werden zunächst in den Dev-Branch zusammengeführt und getestet, bevor sie in die Hauptversion einfließen.

Main-Branch

Nur geprüfte, stabile Änderungen aus dem Dev-Branch werden in den Main-Branch integriert - für maximale Produktionsstabilität.





Modul 1 Zusammenfassung: Git-Befehle im Überblick

git init	Erstellt ein neues Git-Repository
git clone [url]	Kopiert ein bestehendes Repository
git add [datei]	Fügt Änderungen zum Staging-Bereich hinzu
git commit -m "Nachricht"	Speichert Änderungen permanent mit Beschreibung
git branch [name]	Erstellt einen neuen Branch
git checkout [branch]	Wechselt zu einem anderen Branch
git merge [branch]	Führt Änderungen aus einem Branch zusammen
git pull	Aktualisiert lokales Repository
git push	Überträgt lokale Änderungen zum Remote-Repository

Modul 2: GitHub Flow für effiziente Teamarbeit

Branch erstellen

Erstellen Sie einen deskriptiven Feature-Branch von 'main' aus, z.B. 'feature/login-system'

Merge durchführen

Nach Freigabe in den Hauptbranch integrieren



Änderungen vornehmen

Implementieren Sie Ihre Funktionalität mit regelmäßigen Commits

Pull Request öffnen

Initiieren Sie eine Diskussion über Ihre Änderungen

Code Review

Besprechen und verfeinern Sie den Code im Team

KI-optimierte Pull Requests erstellen

KI-generierte PR-Beschreibungen

Nutzen Sie GitHub Copilot, um automatisch aussagekräftige Pull-Request-Beschreibungen zu generieren. Die KI analysiert Ihre Änderungen und fasst sie strukturiert zusammen.

So werden Ihre PRs verständlicher und Reviewprozesse effizienter - besonders wichtig bei der Zusammenarbeit mit 50 Entwicklern.

Automatische Labels & Zuweisungen

Implementieren Sie GitHub Actions, die automatisch passende Labels basierend auf geänderten Dateipfaden, Codemustern oder Commit-Nachrichten zuweisen.

Dies verbessert die Organisation und Priorisierung von Pull Requests in großen Teams und ermöglicht schnellere Reaktionszeiten bei kritischen Änderungen.

Code-Review-Etikette für hohe Commit-Frequenz

Zeitliche Begrenzung

Begrenzen Sie Code-Reviews auf maximal 400 Zeilen pro Sitzung, um Reviewmüdigkeit zu vermeiden. Bei 50 Commits täglich ist Fokus entscheidend.

Konstruktives Feedback

Formulieren Sie Feedback stets lösungsorientiert und respektvoll. Fragen statt Anweisungen fördern den gemeinsamen Lernprozess im Team.

Klare Freigabeprozesse

Definieren Sie eindeutige Kriterien für die Freigabe von Code, um bei hohem Durchsatz schnelle Entscheidungen zu ermöglichen.

Automatisierte Prüfungen

Nutzen Sie Tools für automatische Code-Qualitätsprüfungen, um menschliche Reviewer von Routineaufgaben zu entlasten.

Break-out Session: Bot-gestützte PR-Kommentare

Diskussionsfrage

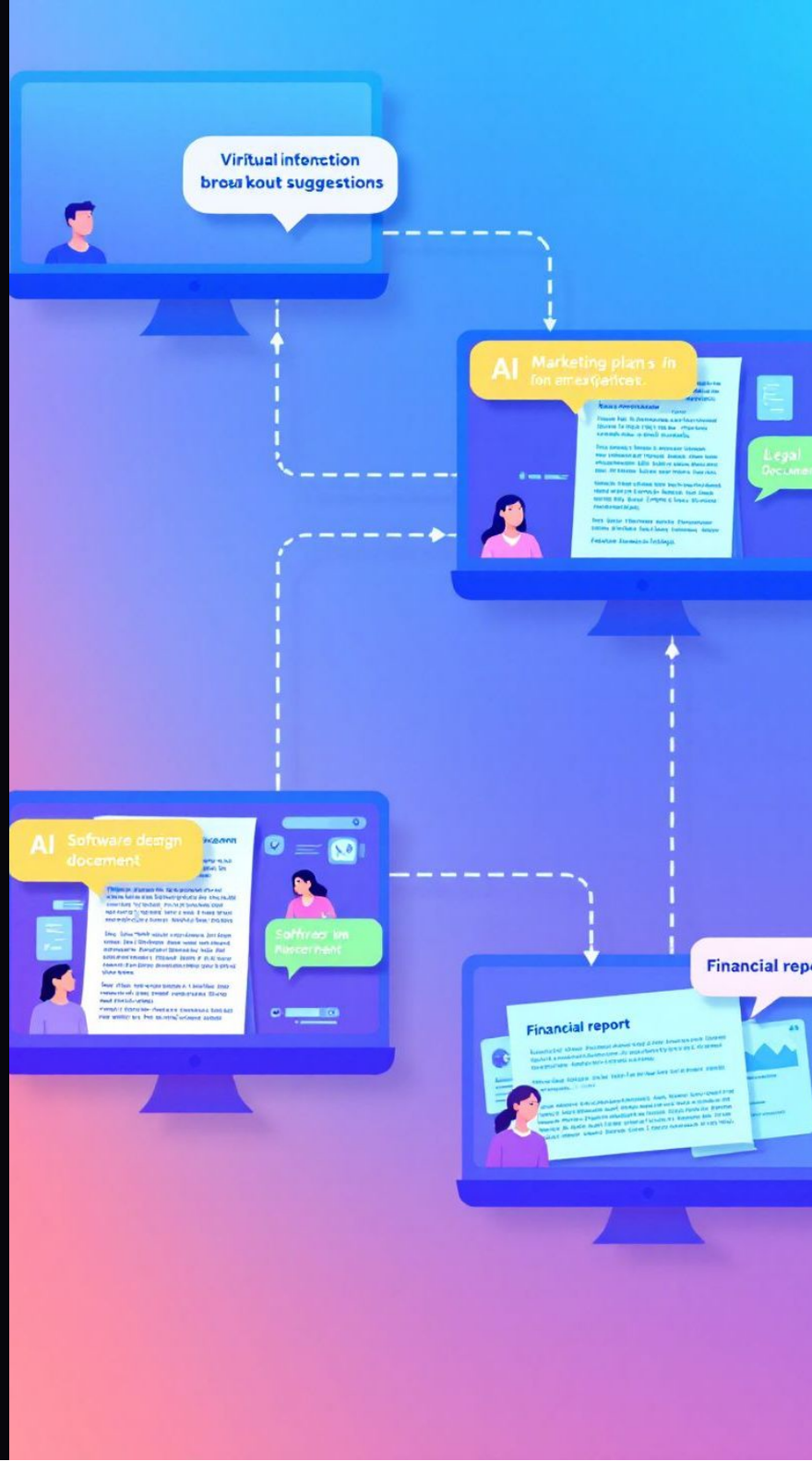
Sollte jeder Pull-Request-Kommentar zunächst von einem KI-Bot vorformuliert werden, um eine konsistente Tonalität und Qualität zu gewährleisten?

Pro-Argumente

- Höhere Konsistenz im Feedback-Stil
- Vermeidung von missverständlichen Formulierungen
- Entlastung der Reviewer bei Routinekommentaren
- Schnelleres Feedback bei hohem Commit-Volumen

Contra-Argumente

- Verlust der persönlichen Note im Feedback
- Risiko von zu generischen Kommentaren
- Mögliche Übersetzungsprobleme bei komplexen Themen
- Erfordert zusätzliche Überprüfung durch Menschen



GitHub Flow: Quick-Quiz und Ressourcen

- Was ist der Hauptunterschied zwischen GitHub Flow und Git Flow?

  GitHub Flow ist ein vereinfachter Workflow mit einem Hauptbranch und Feature-Branches, während Git Flow komplexer ist und verschiedene Branch-Typen wie develop, release und hotfix umfasst. GitHub Flow eignet sich besonders für kontinuierliche Delivery-Umgebungen und kleinere Teams.

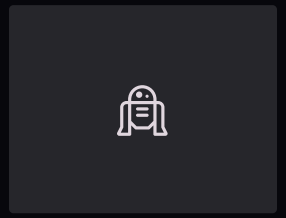
- Warum empfehlen wir Feature-Branches mit beschreibenden Namen?

  Beschreibende Branch-Namen wie 'feature/login-optimierung' oder 'bugfix/memory-leak' machen sofort klar, worum es geht. Dies verbessert die Übersicht in GitHub, erleichtert das Auffinden relevanter Branches und automatisiert Workflows durch Namensmuster.

- Welche GitHub-Funktionen helfen bei der Verwaltung von 50+ täglichen PRs?

  GitHub bietet mehrere Werkzeuge: PR-Templates für strukturierte Einreichungen, automatisches Labeling, CODEOWNERS-Dateien für automatische Reviewer-Zuweisung, Statusprüfungen für CI/CD-Integration und Projektboards zur visuellen PR-Verwaltung nach Status und Priorität.

Modul 3: Die Grundlagen von GitHub Copilot



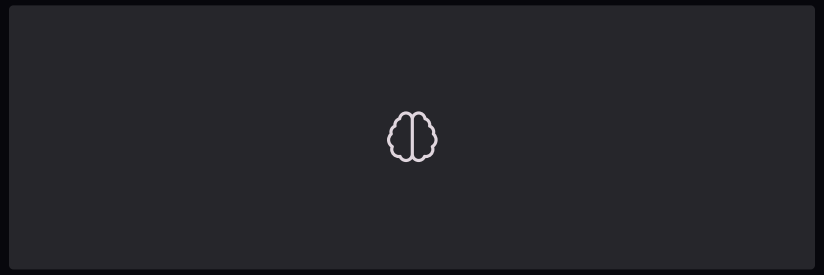
KI-basierte Codegenerierung

Copilot nutzt große Sprachmodelle, um kontextbezogene Code-Vorschläge zu erstellen



Nahtlose IDE-Integration

Direkt in VS Code und anderen Entwicklungsumgebungen verfügbar



Kontext-Verständnis

Analysiert Ihren bestehenden Code, um passende Ergänzungen vorzuschlagen



Produktivitätssteigerung

Reduziert Routineaufgaben und beschleunigt den Entwicklungsprozess

Inline Suggestions vs. Copilot Chat im Vergleich

Inline Suggestions

Copilot bietet automatisch Code-Vorschläge während des Tippens direkt im Editor an. Diese Funktion eignet sich hervorragend für:

- Vervollständigung von Funktionen und Methoden
- Generierung von repetitivem Code
- Vorschläge für typische Code-Muster
- Automatische Vervollständigung von Kommentaren

Copilot Chat

Der interaktive Chat-Modus ermöglicht gezielte Fragen und komplexere Aufgaben. Besonders nützlich für:

- Erklärungen zu bestehendem Code
- Fehlersuche und Debugging-Hilfe
- Konzeptionelle Fragen zur Programmierung
- Generierung komplexerer Code-Strukturen

Die perfekte Prompt-Formel für Entwickler



Ziel (Goal)

Was soll der Code genau erreichen?

Einschränkungen (Constraints)

Welche Technologien, Standards oder Limitierungen gelten?

Tonalität (Tone)

Wie soll der Code strukturiert sein? (kompakt, kommentiert, usw.)

Ein gut strukturierter Prompt führt zu deutlich besseren Ergebnissen. Beispiel: "Erstelle eine JavaScript-Funktion, die einen Text auf Palindrome prüft (Goal). Verwende ES6-Syntax und halte den Code unter 10 Zeilen (Constraints). Füge ausführliche Kommentare zur Funktionsweise hinzu (Tone)."

Live-Demo: JS-Funktion in 30 Sekunden

5	15	10
Prompt-Sekunden	**Generierungs-Sekunden**	**Review-Sekunden**
Formulierung der präzisen Anfrage an Copilot	Automatische Erstellung des Funktionsgerüsts und der Implementierung	Schnelle Überprüfung und minimale Anpassungen

In dieser Live-Demonstration zeigen wir, wie schnell eine vollständige JavaScript-Funktion mit GitHub Copilot erstellt werden kann. Von der Anfrage bis zur fertigen, getesteten Implementierung vergehen nur 30 Sekunden - ein beeindruckendes Beispiel für die Effizienzsteigerung durch KI-gestützte Entwicklung.

Prompt-Optimierungs-Quiz: Was fehlt?

- Prompt 1: "Erstelle eine Funktion zur Dateikonvertierung"

  > Diesem Prompt fehlt die Spezifikation des Dateiformats (Input und Output), die gewünschte Programmiersprache, Fehlerbehandlung und Performance-Anforderungen. Besser wäre: "Erstelle eine Python-Funktion, die CSV-Dateien in JSON konvertiert, mit Fehlerbehandlung für ungültige Einträge und Optimierung für große Dateien."

- Prompt 2: "Schreibe eine API in Node.js mit Express"

  > Hier fehlt die Funktionalität der API, Endpunkte, Authentifizierungsmethode und Datenbankanbindung. Optimiert: "Erstelle eine RESTful API mit Node.js und Express für Benutzerverwaltung mit JWT-Authentifizierung, MongoDB-Anbindung und CRUD-Operationen für Benutzerprofile."

- Prompt 3: "Erstelle einen CSS-Slider"

  > Dieser Prompt benötigt Details zu Design, Funktionalität und Kompatibilität. Verbessert: "Entwickle einen responsiven CSS-Bildslider mit automatischem Wechsel, Navigationspfeilen und Punkten, der in allen modernen Browsern funktioniert und ohne JavaScript auskommt."

Modul 4: Copilot Agent-Mode für komplexe Aufgaben

Agent-Mode aktivieren

Der Agent-Mode ist eine erweiterte Copilot-Funktion, die komplexe Aufgaben selbstständig lösen kann. Aktivieren Sie ihn mit dem Befehl "/agent" im Copilot Chat.

Aufgabe definieren

Beschreiben Sie detailliert, was der Agent erreichen soll - zum Beispiel "Erstelle eine REST API mit Express, die Benutzerdaten in einer MongoDB speichert".

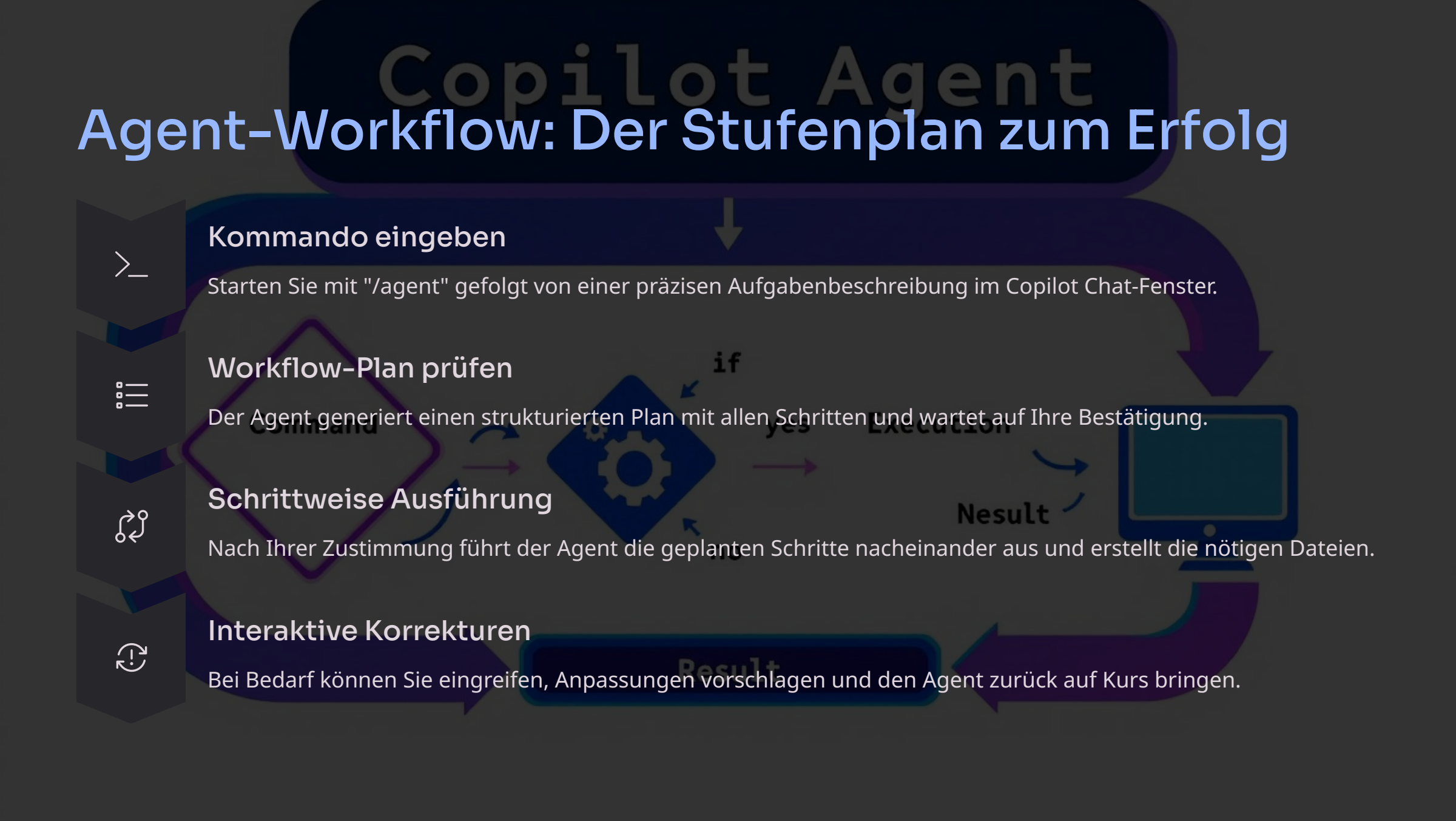
Plan genehmigen

Der Agent erstellt einen detaillierten Arbeitsplan und wartet auf Ihre Genehmigung, bevor er mit der Umsetzung beginnt.

Ergebnisse überprüfen

Kontrollieren Sie die vom Agent generierten Dateien und Strukturen sorgfältig und nehmen Sie bei Bedarf Anpassungen vor.

Agent-Workflow: Der Stufenplan zum Erfolg



Kommando eingeben

Starten Sie mit "/agent" gefolgt von einer präzisen Aufgabenbeschreibung im Copilot Chat-Fenster.

Workflow-Plan prüfen

Der Agent generiert einen strukturierten Plan mit allen Schritten und wartet auf Ihre Bestätigung.

Schrittweise Ausführung

Nach Ihrer Zustimmung führt der Agent die geplanten Schritte nacheinander aus und erstellt die nötigen Dateien.

Interaktive Korrekturen

Bei Bedarf können Sie eingreifen, Anpassungen vorschlagen und den Agent zurück auf Kurs bringen.

Sicherheits-Checkliste für den Agent-Mode



Dry-Run vor Ausführung

Lassen Sie sich immer erst den vollständigen Plan anzeigen und prüfen Sie ihn sorgfältig, bevor Sie die Ausführung bestätigen.



Berechtigungsprüfung

Achten Sie besonders auf Aktionen, die Systemänderungen, Paketinstallationen oder Netzwerkzugriffe beinhalten.



Code-Review vor Integration

Überprüfen Sie generierten Code grundsätzlich auf Sicherheitslücken und potenzielle Probleme.



Isolation wichtiger Umgebungen

Nutzen Sie den Agent nie direkt in Produktionsumgebungen oder mit sensiblen Daten.

Gruppen-Challenge: API mit Self-Healing Tests

Aufgabenstellung

Entwickeln Sie in Teams eine REST-API für eine Aufgabenverwaltung, die bei Testfehlern selbstständig Korrekturen vorschlägt und implementiert.

Vorgehensweise

- API-Endpunkte mit Copilot Agent generieren lassen
- Test-Suite mit automatischen Fehlerkorrektur-Vorschlägen erstellen
- Feedback-Schleife zwischen Tests und Code implementieren
- Dokumentation des Self-Healing-Prozesses anfertigen

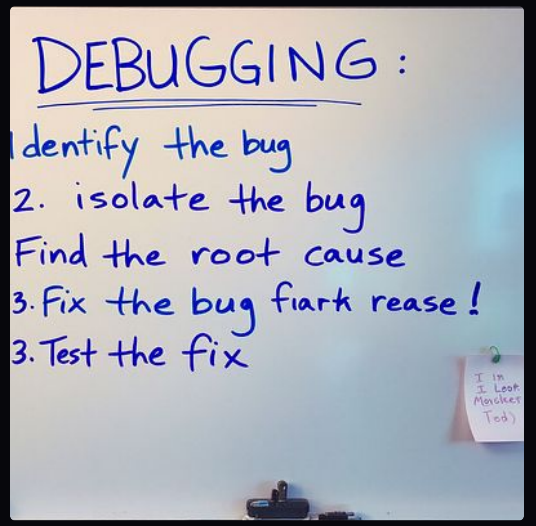
Erfolgskriterien

- API besteht alle Funktionstests
- Mindestens drei automatische Korrekturen dokumentiert
- Implementierung folgt REST-Best-Practices
- Code-Qualität durch Linting-Tools bestätigt

Agent-Mode: Fail-Stories und Lehren









In dieser Zusammenfassung sammeln wir gemeinsam Erfahrungen mit dem Agent-Mode. Jedes Team teilt eine "Fail-Story", bei der der Agent unerwartete oder problematische Ergebnisse lieferte. Aus diesen Fehlschlägen leiten wir wertvolle Lektionen für die effektivere Nutzung ab und erstellen eine gemeinsame Wissensbasis.

Modul 5: VS Code Power User werden



Tastaturkürzel

Beschleunigen Sie Ihren Workflow durch Shortcuts für häufige Aktionen und navigieren Sie effizient durch Code.



Layout-Optimierung

Konfigurieren Sie Ihr Arbeitsumfeld mit Split-Editoren, Zen-Modus und anpassbaren Panels.



Erweiterungen

Erweitern Sie VS Code mit spezialisierten Tools für Ihre Programmiersprachen und Workflows.

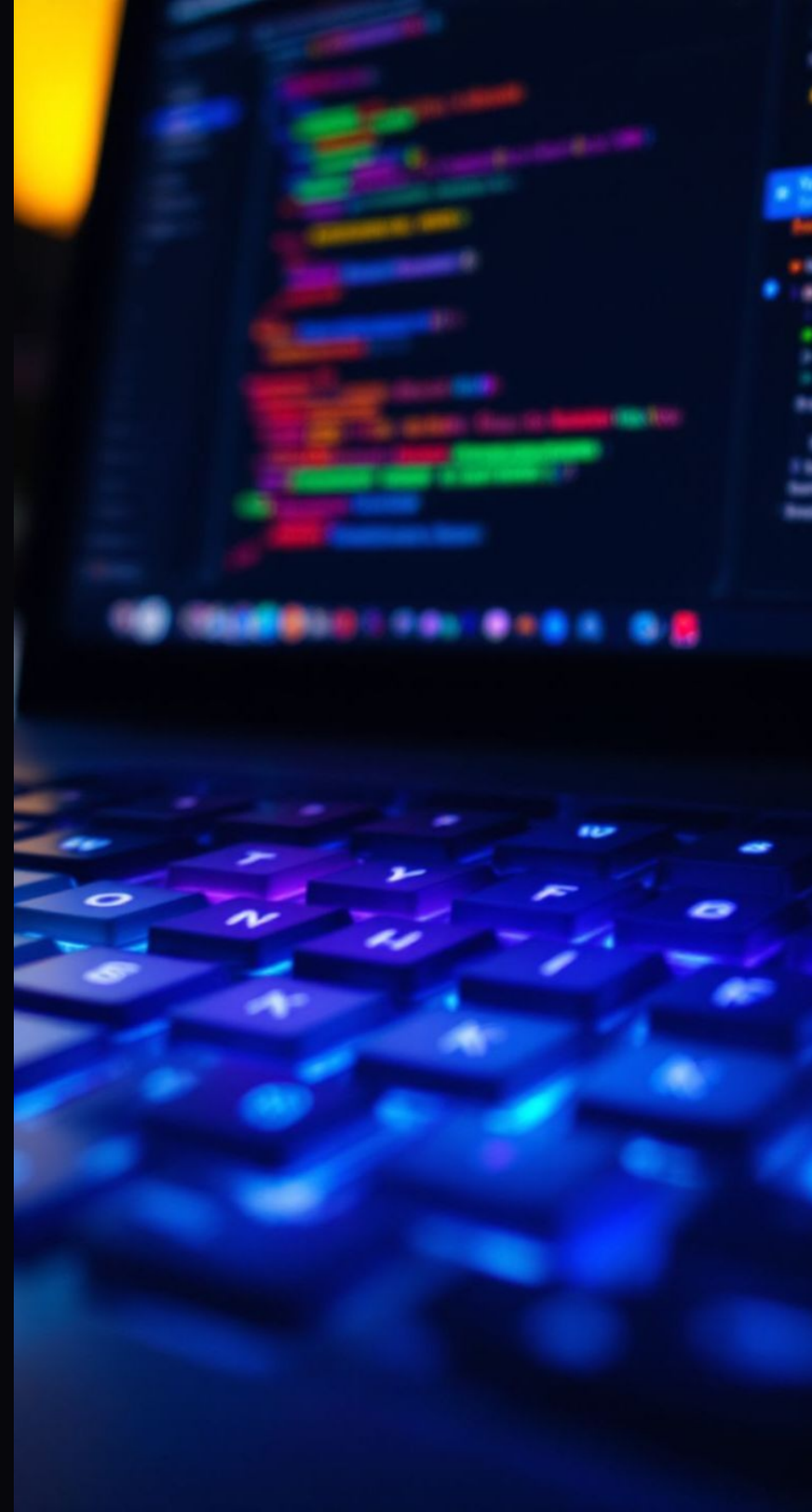


Personalisierung

Passen Sie Themes, Schriftarten und Einstellungen an Ihre Bedürfnisse an.

Die Top 10 VS Code Shortcuts im Überblick

Ctrl+P / Cmd+P	Schnelle Dateisuche
Ctrl+Shift+P / Cmd+Shift+P	Befehlspalette öffnen
Alt+↑/↓ / Option+↑/↓	Zeilen verschieben
Ctrl+D / Cmd+D	Nächstes Vorkommen auswählen
Ctrl+/ / Cmd+/	Zeile kommentieren/entkommentieren
Ctrl+` / Cmd+`	Terminal öffnen/schließen
F12	Zur Definition springen
Ctrl+Space / Cmd+Space	IntelliSense aktivieren
Ctrl+\ / Cmd+\	Editor splitten
Ctrl+B / Cmd+B	Seitenleiste ein-/ausblenden



Multi-Cursor und KI-Snippets kombinieren

Multi-Cursor-Techniken

Mit mehreren Cursoren gleichzeitig zu arbeiten ist eine der leistungsstärksten Funktionen von VS Code:

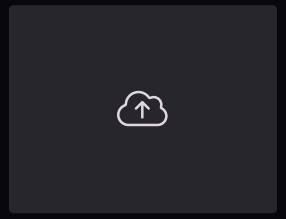
- Alt+Klick / Option+Klick für zusätzliche Cursor
- Ctrl+Alt+↑/↓ / Cmd+Option+↑/↓ für Cursor in Zeilen darüber/darunter
- Ctrl+D / Cmd+D zum Auswählen des nächsten Vorkommens
- Alt+Shift+I / Option+Shift+I für Cursor am Ende jeder ausgewählten Zeile

KI-generierte Snippets

Kombinieren Sie Multi-Cursor mit Copilot für maximale Effizienz:

- Erstellen Sie ein Snippet-Template mit Platzhaltern
- Setzen Sie mehrere Cursor an den Platzhaltern
- Lassen Sie Copilot für jeden Cursor passende Vorschläge generieren
- Passen Sie die generierten Snippets bei Bedarf an

Settings Sync für konsistente Teamumgebungen



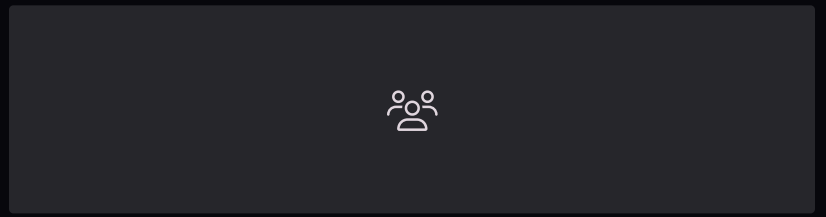
Einrichtung der Synchronisation

Aktivieren Sie Settings Sync mit GitHub-Konto



Auswahl der Einstellungen

Wählen Sie zu synchronisierende Elemente wie Einstellungen, Snippets und Erweiterungen



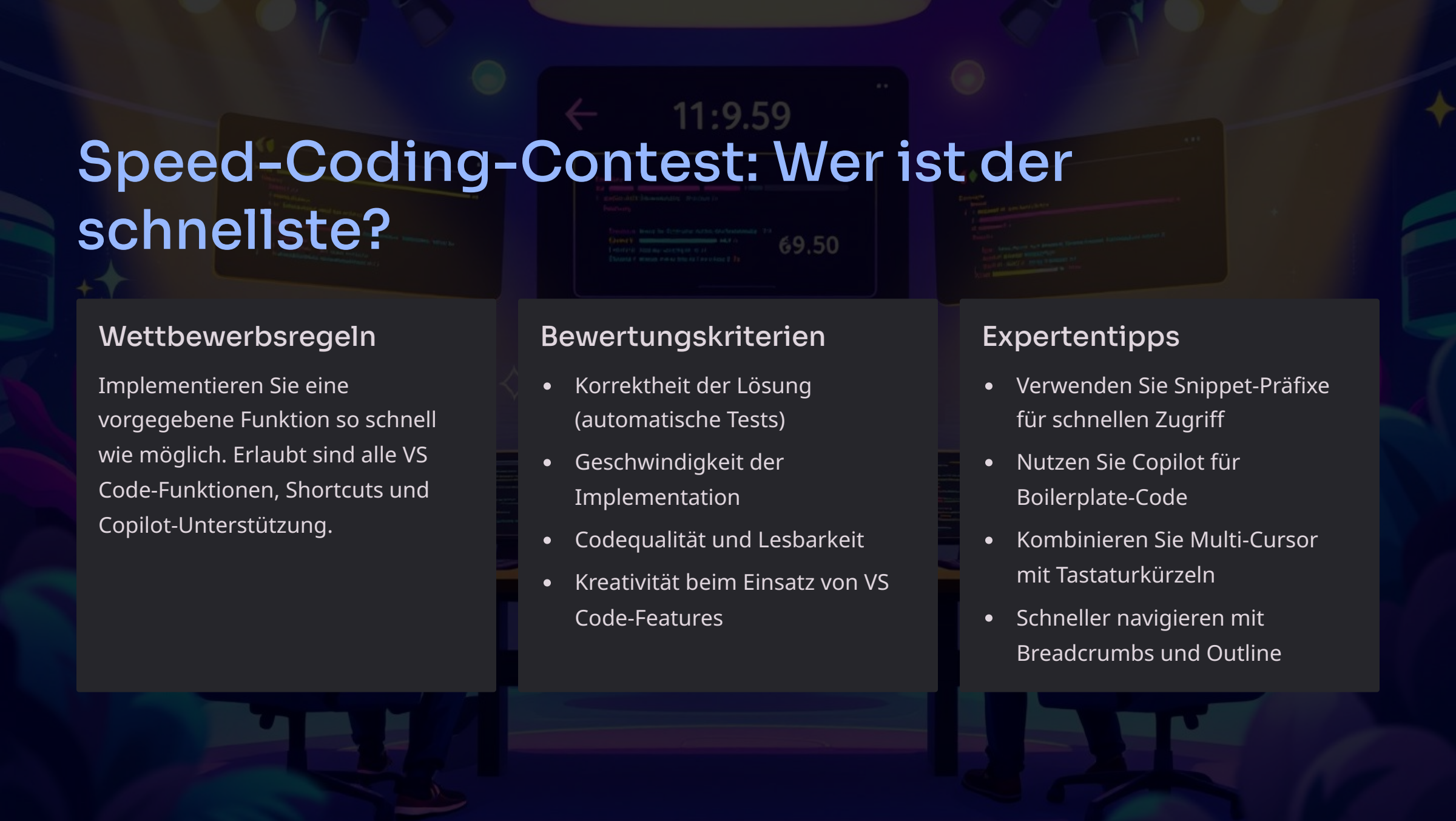
Team-Templates

Erstellen Sie gemeinsame Konfigurationsprofile für Ihr Team



Datenschutz-Management

Konfigurieren Sie sensible Einstellungen, die nicht synchronisiert werden sollen



Speed-Coding-Contest: Wer ist der schnellste?

Wettbewerbsregeln

Implementieren Sie eine vorgegebene Funktion so schnell wie möglich. Erlaubt sind alle VS Code-Funktionen, Shortcuts und Copilot-Unterstützung.

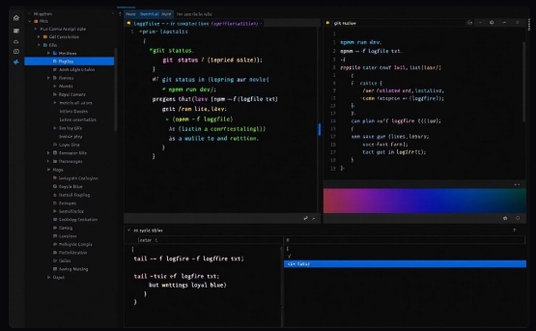
Bewertungskriterien

- Korrektheit der Lösung (automatische Tests)
- Geschwindigkeit der Implementation
- Codequalität und Lesbarkeit
- Kreativität beim Einsatz von VS Code-Features

Expertentipps

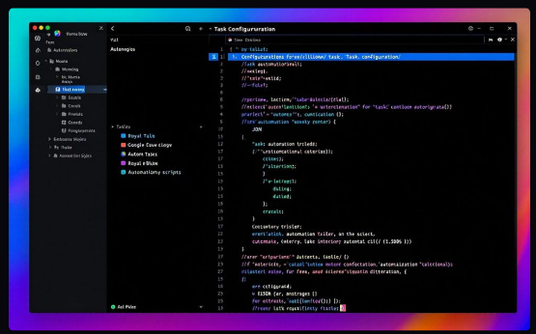
- Verwenden Sie Snippet-Präfixe für schnellen Zugriff
- Nutzen Sie Copilot für Boilerplate-Code
- Kombinieren Sie Multi-Cursor mit Tastaturkürzeln
- Schneller navigieren mit Breadcrumbs und Outline

Modul 6: Terminal-Integration und Aufgabenautomation



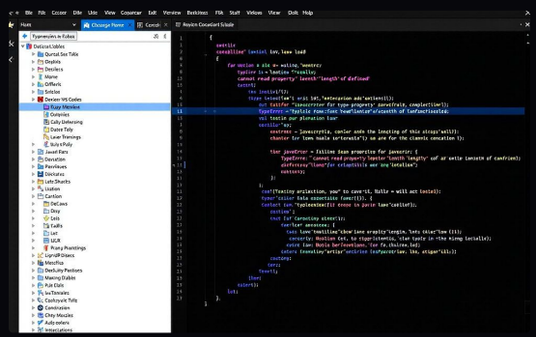
Integriertes Terminal

Nutzen Sie das in VS Code eingebaute Terminal für nahtloses Arbeiten ohne Kontextwechsel. Mehrere Terminal-Instanzen können parallel geöffnet und organisiert werden.



Aufgabenautomatisierung

Erstellen Sie benutzerdefinierte Tasks für wiederkehrende Entwicklungsaufgaben wie Build-Prozesse, Tests oder Deployments.



Problemerkennung

VS Code kann automatisch Fehler und Warnungen in der Ausgabe von Aufgaben erkennen und direkt zu den entsprechenden Codezeilen navigieren.

Split-Pane Terminal Race: Wer baut zuerst?

Terminal-Setup

Teilen Sie Ihr integriertes Terminal in drei Bereiche auf: einen für Git-Operationen, einen für Build-Prozesse und einen für Tests.

Repository klonen

Klonen Sie das vorgegebene Projekt-Repository mit optimierten Git-Befehlen für maximale Geschwindigkeit.

Abhängigkeiten installieren

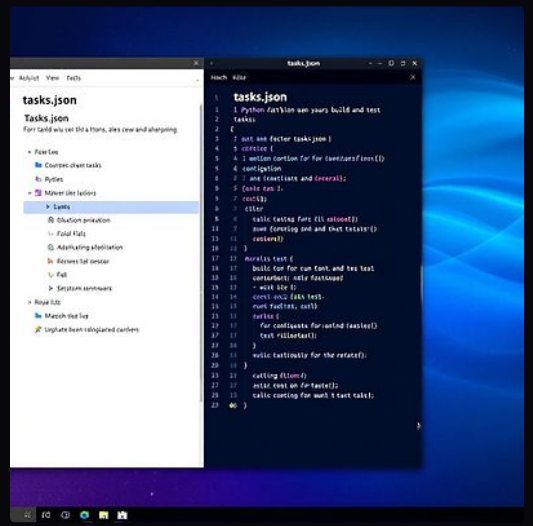
Verwenden Sie parallele Installation und Caching-Strategien, um die Einrichtungszeit zu minimieren.

Build und Verifizierung

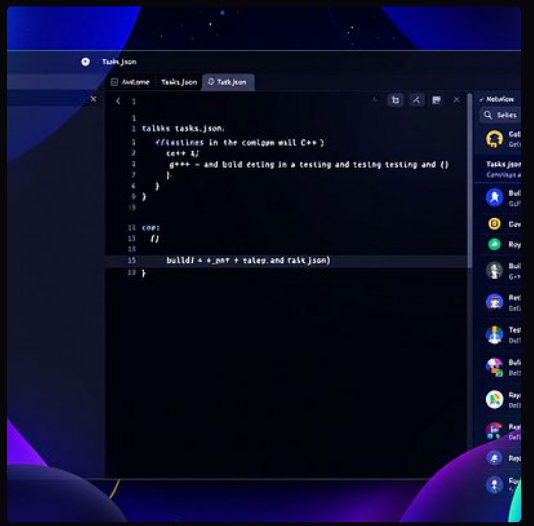
Führen Sie den Build-Prozess und die anschließenden Tests durch, um die erfolgreiche Einrichtung zu bestätigen.

Optimierte tasks.json Templates für Ihre Projekte









Unsere Task-Template-Gallery bietet vorkonfigurierte tasks.json Dateien für verschiedene Projekttypen. Verwenden Sie diese als Ausgangspunkt für Ihre eigenen Automatisierungen. Jedes Template enthält Best Practices für Ihre spezifische Entwicklungsumgebung und kann einfach an Ihre Bedürfnisse angepasst werden.

Umgebungsvariablen und Secrets sicher verwalten

Lokale Umgebungsvariablen

Für die lokale Entwicklung können Sie Umgebungsvariablen direkt in der launch.json und tasks.json konfigurieren:

- Variablen in "env" Abschnitt definieren
- .env-Dateien in Projektverzeichnis nutzen
- Node.js dotenv-Paket für automatisches Laden

Sichere Geheimnisverwaltung

Für sensible Daten wie API-Schlüssel verwenden Sie besser die integrierten Sicherheitsmechanismen:

- Nie Secrets in die Versionskontrolle einchecken
- VS Code Secrets Extension für verschlüsselte Speicherung
- Nutzung des Betriebssystem-Schlüsselbunds
- Separate Entwicklungs- und Produktionsschlüssel

Shell-Präferenzen: Welche ist Ihr Favorit?

PowerShell

Mächtige Windows-Shell mit Objekt-Pipeline und .NET-Integration, ideal für Automatisierung auf Windows-Systemen.

Bash

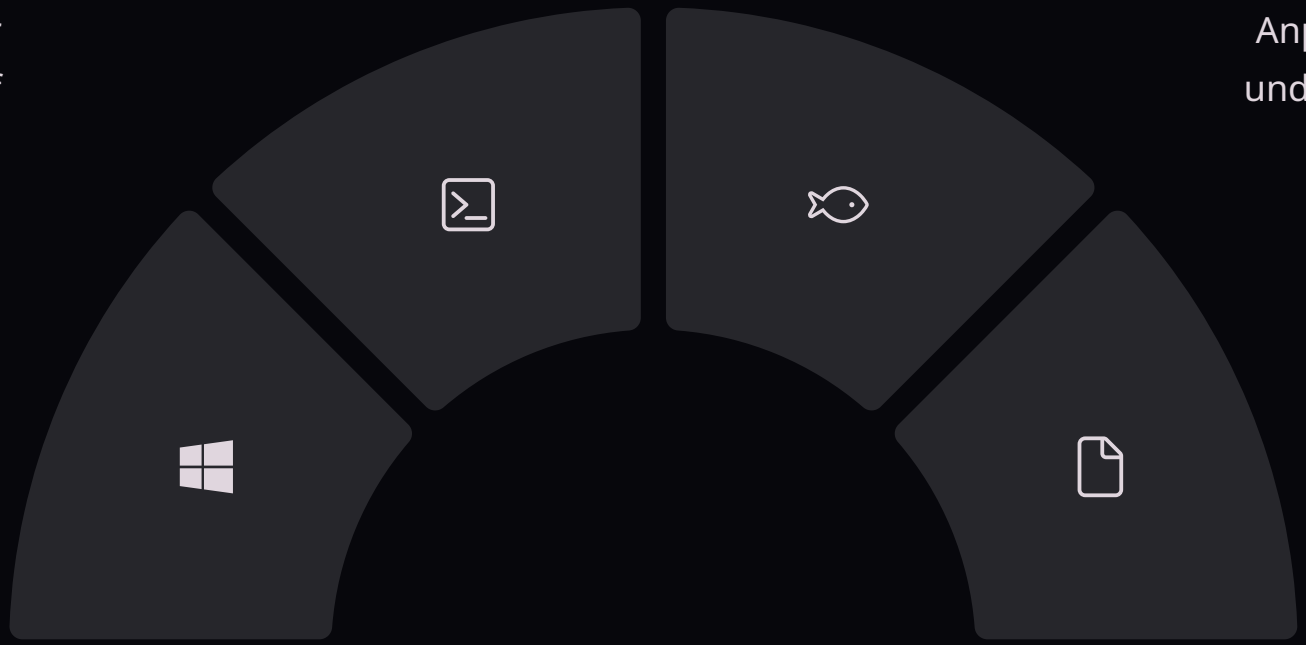
Standard auf Linux und macOS mit umfangreicher Skriptunterstützung und breitem Ökosystem an Tools.

Fish

Benutzerfreundliche Shell mit intelligenter Autovervollständigung und syntax-Highlighting ohne Konfigurationsaufwand.

Zsh

Erweiterte Bash-Alternative mit besseren Anpassungsmöglichkeiten und dem populären Oh-My-Zsh-Framework.



Modul 7: Debugging-Masterclass mit VS Code

Integrierter Debugger

VS Code bietet einen leistungsstarken, sprachübergreifenden Debugger für nahtloses Debugging direkt in der IDE.

Intelligente Breakpoints

Setzen Sie Haltepunkte mit Bedingungen, Protokollierung und Trefferanzahl für präzises Debugging.

Variable Inspektion

Untersuchen Sie Variablenwerte, Objektstrukturen und den Aufrufstapel in Echtzeit während der Ausführung.

Hot Reloading

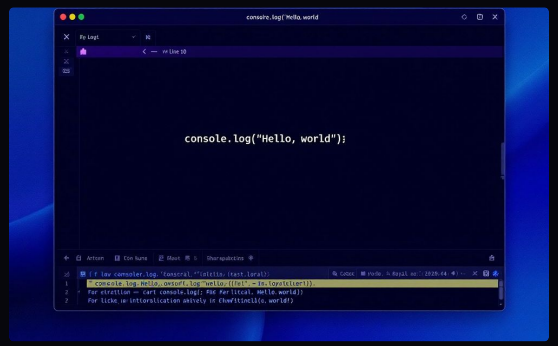
Ändern Sie Code während des Debugging-Prozesses und sehen Sie sofort die Auswirkungen ohne Neustart.

Breakpoint-Arten optimal einsetzen



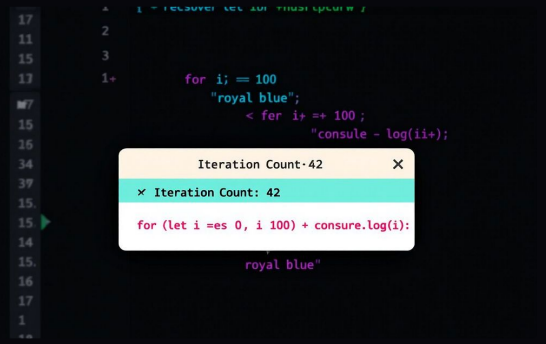
Bedingte Breakpoints

Halten Sie die Ausführung nur an, wenn eine bestimmte Bedingung erfüllt ist. Ideal für Fehler, die nur unter speziellen Umständen auftreten, wie bei bestimmten Eingabewerten oder Zuständen.



Log-Points

Protokollieren Sie Informationen ohne die Ausführung zu unterbrechen. Perfekt für die Überwachung von Variablenwerten in Schleifen oder bei zeitkritischen Prozessen, bei denen ein regulärer Breakpoint problematisch wäre.



Hit-Count Breakpoints

Pausieren Sie erst nach einer bestimmten Anzahl von Durchläufen. Besonders nützlich bei der Fehlersuche in Schleifen, wenn das Problem erst nach mehreren Iterationen auftritt.

Python-Debugging mit interaktivem Fenster

Interactive Window Setup

Das Python Interactive Window in VS Code kombiniert die Leistung von Jupyter Notebooks mit der Flexibilität der IDE:

- Zellen mit #%% Kommentaren definieren
- Code blockweise ausführen mit "Run Cell"
- Variablen im Variables Explorer beobachten
- Inline-Plots und -Diagramme anzeigen

Live-Inspektion

Während der Ausführung können Sie:

- Variablen in Echtzeit inspizieren und modifizieren
- Zusätzliche Befehle im laufenden Kontext ausführen
- DataFrames und komplexe Objekte in tabellarischer Form anzeigen
- Den Ausführungskontext zwischen Zellen beibehalten



Node.js Entwicklung mit Hot-Reload

Hot-Reload Setup

Richten Sie automatisches Neuladen für Ihre Node.js-Anwendung ein, um die Entwicklungsgeschwindigkeit zu erhöhen.

Debugger-Anbindung

Konfigurieren Sie VS Code für die Verbindung mit dem Node.js-Debugger, auch bei laufender Anwendung.

Chrome DevTools Integration

Verbinden Sie den VS Code-Debugger mit Chrome für umfassende Frontend- und Backend-Fehlerbehebung.

Persistente Breakpoints

Behalten Sie Ihre Breakpoints auch nach dem Hot-Reload bei, um komplexe Fehler konsistent zu verfolgen.

Team-Challenge: 15-Minuten Bug Bash

Challenge-Format

In dieser intensiven 15-minütigen Session arbeitet jedes Team zusammen, um so viele Bugs wie möglich in einem vorgegebenen Projekt zu finden und zu beheben.

Vorgehensweise

- Jedes Team erhält das gleiche fehlerbehaftete Repository
- Einsatz aller gelernten Debug-Techniken
- Copilot zur schnellen Fehleranalyse nutzen
- Fixes committen und dokumentieren

Bewertung

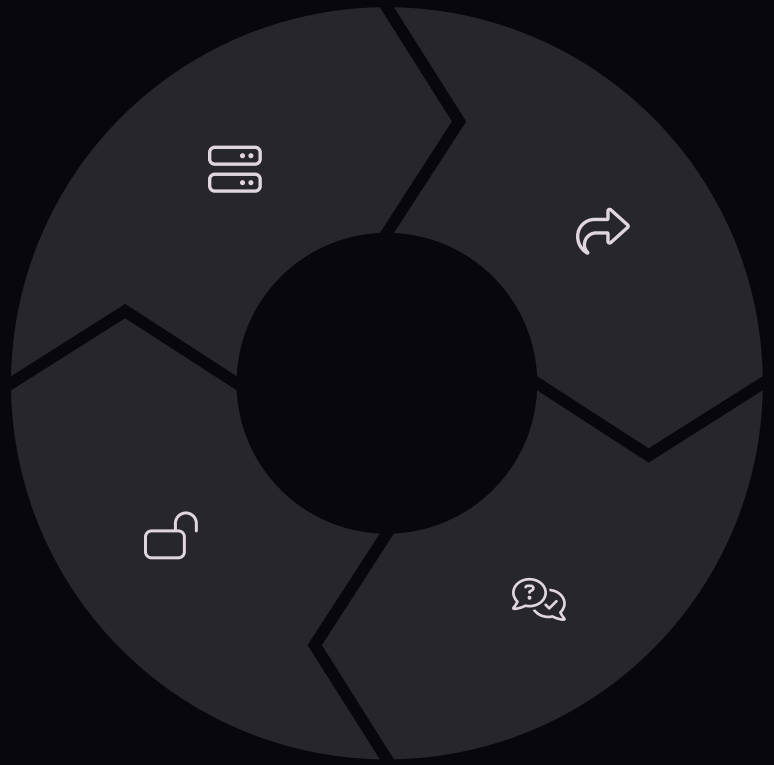
- Anzahl der gefundenen und behobenen Bugs
- Qualität der Fixes (keine neuen Bugs)
- Dokumentation der Fehlerursachen
- Teamwork und Aufgabenverteilung



Modul 8: Live Collaboration mit VS Code

Live Server

Lokaler Entwicklungsserver mit automatischem Reload bei Änderungen



Live Share

Echtzeit-Kollaboration mit Teammitgliedern im gleichen Code

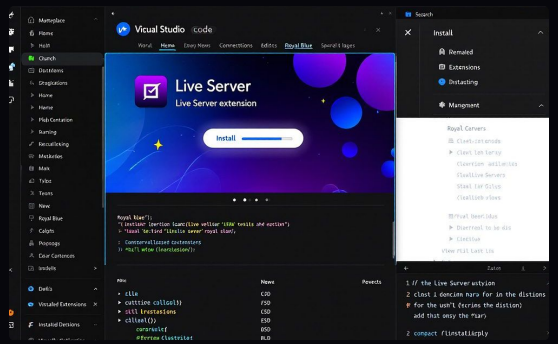
Berechtigungsverwaltung

Präzise Kontrolle über Zugriffsrechte und Sichtbarkeit

Chat & Audio

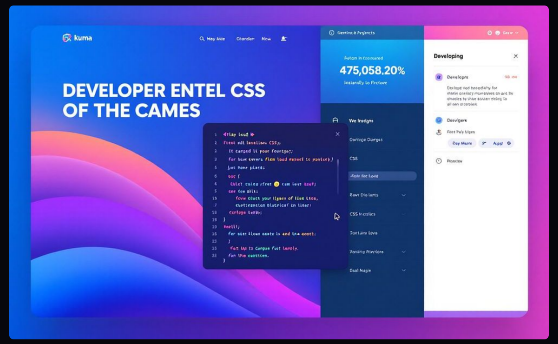
Integrierte Kommunikationstools für nahtlose Zusammenarbeit

CSS-Hot-Reload mit Live Server in Aktion



Live Server Extension

Installieren Sie die Live Server Extension über den VS Code Marketplace für sofortigen Zugriff auf die Hot-Reload-Funktionalität.



Instant Preview

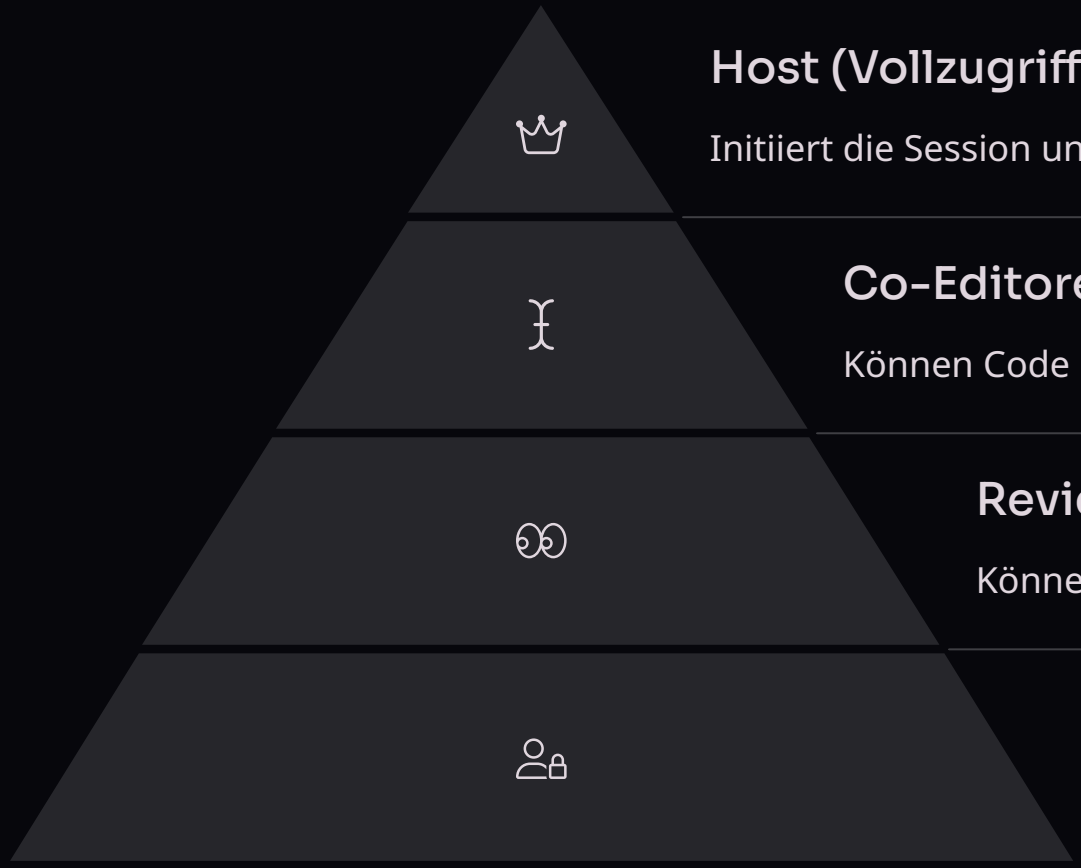
Sehen Sie Ihre CSS-Änderungen in Echtzeit im Browser, ohne manuelles Neuladen. Perfekt für schnelles Design-Prototyping und Feinabstimmung.



Multi-Browser-Testing

Testen Sie Ihr responsives Design gleichzeitig in verschiedenen Browsern und Bildschirmgrößen für konsistente Ergebnisse.

Live Share für große Teams organisieren



Host (Vollzugriff)

Initiiert die Session und hat alle Rechte

Co-Editoren (Schreibzugriff)

Können Code bearbeiten und Debugging durchführen

Reviewer (Lesezugriff)

Können Code einsehen, aber nicht ändern

Datei-spezifische Berechtigungen

Granulare Kontrolle für bestimmte Dateien

Bei 50 Teilnehmern ist eine klare Berechtigungsstruktur entscheidend. Überlegen Sie, die Gruppen in separate Sessions aufzuteilen oder spezifische "Beobachter"- und "Beitragende"-Rollen zuzuweisen. Für große Sessions sollten Sie auch Bandbreitenlimits und Performance-Optimierungen berücksichtigen.

Pair-Debugging mit Driver/Navigator-Rotation



Driver

Hat die Tastaturkontrolle und implementiert aktiv Code und Änderungen nach Anleitung des Navigators.



Navigator

Leitet den Prozess, denkt strategisch und gibt Anweisungen zur Problemlösung ohne selbst zu tippen.



Rotation

Regelmäßiger Wechsel der Rollen nach festgelegten Intervallen (10-15 Minuten) für optimale Lerneffekte.

Collaborative Development: Best Practices

1

Kommunikation

Klare Absprachen über Zuständigkeiten und regelmäßige Status-Updates, auch bei asynchroner Arbeit

2

Dokumentation

Gemeinsame Coding-Standards und aktualisierte README-Dateien für alle Teilnehmer

3

Werkzeuge

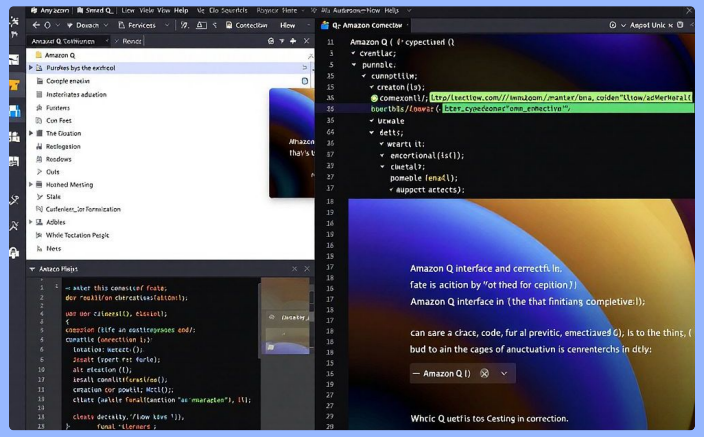
Einheitliche Entwicklungsumgebungen mit synchronisierten Einstellungen und Extensions

4

Prozesse

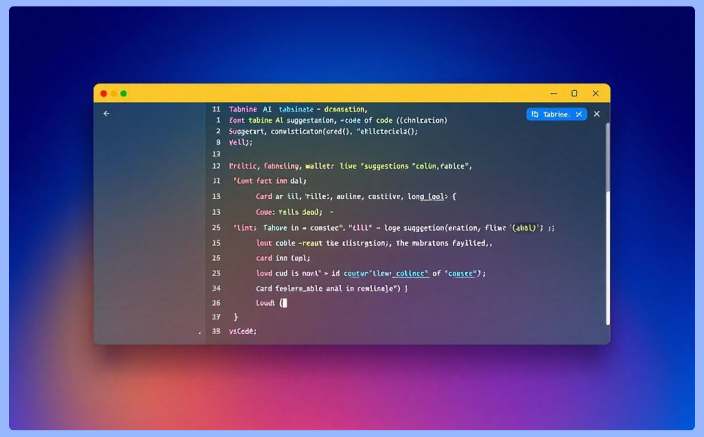
Strukturierte Code-Review-Zyklen und klare Merge-Richtlinien für konsistente Qualität

Modul 9: KI-Tools jenseits von Copilot



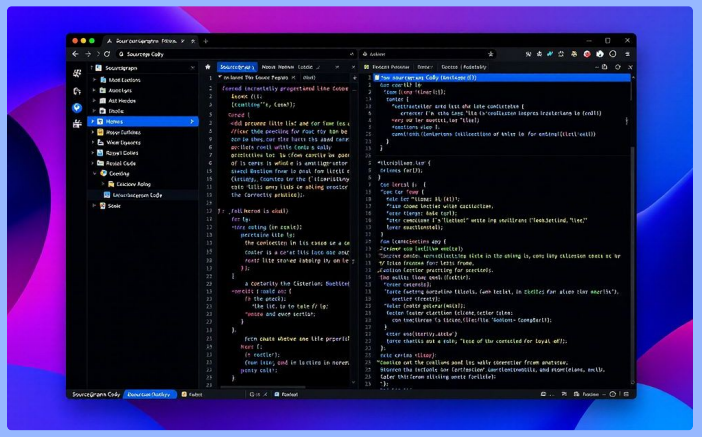
Amazon Q

Amazons KI-Assistent für Entwickler mit tiefer AWS-Integration und unternehmensfreundlichen Datenschutzfunktionen.



Tabnine

Fokussiert auf lokale KI-Modelle, die vollständig auf Ihrem System laufen können - ideal für sicherheitsbewusste Umgebungen.



Cody by Sourcegraph

Spezialisiert auf Code-Verständnis und Navigation in großen Codebasen mit fortschrittlicher Semantikanalyse.

Feature-Vergleich: KI-Coding-Assistenten

Feature	GitHub Copilot	Amazon Q	Tabnine	Cody
Codevorschläge	Sehr gut	Gut	Gut	Gut
Chat-Interface	Ja	Ja	Eingeschränkt	Ja
Agent-Modus	Ja	Nein	Nein	Teilweise
Lokale Modelle	Nein	Nein	Ja	Teilweise
Cloud-Abhängigkeit	Hoch	Hoch	Niedrig	Mittel
Sprachunterstützung	Sehr breit	Breit	Breit	Mittel
Enterprise-Features	Gut	Sehr gut	Gut	Mittel

Datenschutz- und Lizenzprüfung für Unternehmen

- Welche Daten werden an KI-Anbieter übermittelt?

  Die meisten KI-Coding-Assistenten senden Ihren Code zur Analyse an Cloud-Server. GitHub Copilot überträgt Codeschnipsel, Kommentare und geöffnete Dateien. Amazon Q bietet spezielle Enterprise-Versionen mit Datenverbleib in Ihrer AWS-Region. Tabnine kann vollständig lokal betrieben werden, sendet standardmäßig aber anonymisierte Nutzungsdaten.

- Wie steht es um die Trainings- und Verwendungsrechte?

  Die AGB unterscheiden sich erheblich. GitHub behält sich das Recht vor, übermittelte Daten zum Training zu verwenden, bietet aber Enterprise-Kunden Opt-out-Optionen. Amazon Q garantiert, dass Kundendaten nicht zum Training verwendet werden. Bei der Auswahl einer Lösung ist eine gründliche Prüfung der Lizenzvereinbarungen mit Ihrer Rechtsabteilung essentiell.

- Welche Compliance-Anforderungen müssen beachtet werden?

  Für Unternehmen in regulierten Branchen oder mit DSGVO-Anforderungen sind Aspekte wie Datenspeicherungsorte, Verschlüsselung, Löschrichtlinien und Audit-Trails entscheidend. Einige Tools bieten spezielle Compliance-Modi oder Enterprise-Versionen mit erweiterten Sicherheitsfunktionen wie private Instanzen oder VPC-Integration.

A/B-Test: Welches KI-Tool liefert den besten Fix?

Experiment-Setup

Jedes Team erhält die gleiche fehlerhafte Codebasis und wird in vier Gruppen aufgeteilt. Jede Gruppe nutzt einen anderen KI-Assistenten, um die identifizierten Probleme zu beheben:

- Gruppe A: GitHub Copilot
- Gruppe B: Amazon Q
- Gruppe C: Tabnine
- Gruppe D: Cody by Sourcegraph

Bewertungskriterien

Die Lösungen werden anhand objektiver Metriken verglichen:

- Korrektheit (automatisierte Tests)
- Code-Qualität (Linting-Scores)
- Effizienz (Performance-Benchmarks)
- Zeitaufwand bis zur Lösung
- Anzahl der nötigen Prompt-Iterationen

Kursabschluss: Zertifikate und nächste Schritte



Zertifizierung

Erhalten Sie Ihr offizielles KI-gestützter Entwickler Zertifikat

Abschlussprojekt

Wenden Sie alle gelernten Techniken in einer Teamaufgabe an

Weiterbildungspfad

Entdecken Sie fortgeschrittene Lernressourcen für Ihre Entwicklung

Community

Bleiben Sie mit Gleichgesinnten über unsere Plattform verbunden

Herzlichen Glückwunsch zum erfolgreichen Abschluss des VS Code & Git Meisterkurses! Ihr neu erworbenes Wissen über KI-gestützte Entwicklung wird Ihnen einen erheblichen Produktivitätsvorteil in Ihrer täglichen Arbeit verschaffen. Wir freuen uns, Sie auf Ihrem weiteren Weg zu begleiten.